SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Timed Online Assignment – January Semester 2022

Structured Programming

Thursday, 19 May 2022 **1:00 pm – 3:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **SIX (6)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, Your Name, and SUSS PI No. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all the questions. (Total 100 marks)

You can use ONLY built-in functions and packages covered in the module, e.g., math, random. You can use ONLY built-in types. Do not write classes.

Question 1

Express a sequence of statements based on computational logic.

Write code segments for the following.

(a) Given a string, display `valid` if the length of the string is 8 letters, of which the first 3 must be uppercase letters, followed by 4 digits and the last letter ends with one of the letters 'S', 'X', or 'Z'. Otherwise, display `invalid`.

(10 marks)

(b) Given a positive integer n less than 10, display a n by n multiplication table. For example, if n is 5, display the following:

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25
```

(15 marks)

Question 2

Employ structured programming principles to develop a program.

A car park has a grace period of 10 minutes, after which the charge is $1.50 for the first hour or part thereof of parking and 2 cents per minute after the first hour. The maximum charge however, is $5.00.

Some examples of the parking charges are:

- 9 minutes, no charge.
- 35 minutes, charge is $1.50.
- 1 hour 10 minutes, charge is $1.70.
- 6 hours, charge is $5.00.

A program continually reads in a float value representing a duration and displays the car park charge. Input is a value in the format `h:mm`, where h is the hour and mm the minutes. Assume input will be in the correct format. The program ends when user presses just the <Enter> key at the prompt.

A program execution sample is as follows:

```
Enter duration: 0:15
15 mins parking. Charge is $1.50
Enter duration: 0:07
7 mins parking. No charge
Enter duration: 1:12
1 hr 12 mins parking. Please pay $1.74
Enter duration: 5:00
5 hr 0 mins parking. Please pay $5.00
Enter duration: <Enter>
```

(a) Write a function `getDurationInMinutes(duration)` where the parameter duration is a string in the format h:mm. The function computes and returns the duration in minutes.

(5 marks)

(b) Write a function `computeCharge(minutes)` where the parameter is an integer representing the total minutes parked. The functions calculates and returns the parking charge.

(10 marks)

(c) Write the program. Make use of the functions in Q2(a) and (b).

(10 marks)

Question 3

Solve computational problems using structured programming and apply data structures in problem solving.

(a) Given the following list:

```
qtyPrice = [2, 1.5, 4, 0.5, 1, 2.5, 3, 2.0]
```

Each pair of value in the list represents a quantity and unit price. For example, the first pair 2, 1.5 is 2 units at 1.5 per unit. Write a code segment using control structures to display the subtotal and total price as follows:

```
2 x 1.5 = 3.0
4 x 0.5 = 2.0
1 x 2.5 = 2.5
3 x 2.0 = 6.0
4 items.
Total price $13.50
```

(15 marks)

(b) Given the following declaration,

```
numlist = []
```

Write a code segment that repeatedly prompts for integer values and inserts the value to `numlist`. Program ends when the length of `numlist` is 5. The value must be added in ascending order. Do not use any sort functions. Display the contents of the list after each input. A sample execution of the program is as follows:

```
Enter num: 5
[5]
Enter num: 9
[5,9]
Enter num: 5
[5,5,9]
Enter num: 2
[2,5,5,9]
Enter num: 7
[2,5,5,7,9]
```

(10 marks)

Question 4

Solve computational problems using structured programming, apply data structures and file handling.

A media screening room has rows of seats. The rows are labelled A, B, C, etc and seat numbers 1, 2, 3 etc. A seating plan for 3 rows, 4 seats per row is as follows:

```
A O X O X
B O O O X
C O X O O
  1 2 3 4
```

Empty seats are marked with `O`, while those occupied marked with `X`.

(a) A seating plan with some seats occupied is stored in a file, `seating.txt` as follows:

```
A,O,X,O,X
B,O,O,O,X
C,O,X,O,O
```

Write a code segment to read the file and store the seating plan in a dictionary. The dictionary structure after reading the file should be as follows:

```
seatingPlan = {'A':['O','X','O','X'], 'B':['O','O','O','X'],
                                 'C':['O','X','O','O'] }
```

(8 marks)

(b) Write a code segment to display the seating plan based on the dictionary structure in Q4(a):

```
A O X O X
B O O O X
C O X O O
  1 2 3 4
```

(8 marks)

(c) Write a code segment to allow for booking of seats. To book seats, the user inputs a seat by row and seat number, followed by number of seats to book. Assume all input to be valid. If the seats are available, update the dictionary and display a message. The following sample shows the scenarios to consider:

<u>Sample 1</u> Enter seat no: **B1** Enter number of seats to book: **2** Seats successfully allocated	Assume seat no is valid. No validation required. Update the dictionary with X for seats allocated.
<u>Sample 2</u> Enter seat no: **C1** Enter number of seats to book: **2** Not available	Not available because C2 is occupied.
<u>Sample 3</u> Enter seat no: **C3** Enter number of seats to book: **5** Not available	Not available because of insufficient seats.

(9 marks)

----- END OF PAPER -----